

VC_HW1_314581003

Homework #1 - Color Transform

一、 題目要求：將 lena.png 轉換為三種色彩空間：

- RGB
- YUV
- YCbCr

並輸出 8 張灰階影像，分別為：

- R, G, B
- Y, U, V
- Cb, Cr

二、 色彩空間轉換公式與實作：

- RGB -> YUV：

$$Y = 0.299R + 0.587G + 0.114B$$

$$U = -0.169R - 0.331G + 0.5B + 128$$

$$V = 0.5R - 0.419G - 0.081B + 128$$

- RGB -> YCbCr：

$$\begin{bmatrix} Y \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} 0.257 & 0.504 & 0.098 \\ -0.148 & -0.291 & 0.439 \\ 0.439 & -0.368 & -0.071 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 16 \\ 128 \\ 128 \end{bmatrix}$$

三、 程式碼實作：

1. 讀取圖片：使用 OpenCV 的 `cv2.imread("lena.png")` 讀取影像，

得到 BGR 格式的三維陣列。為了避免誤差，將數據轉為 float32 型別以利運算。

```
# 建立輸出資料夾存放八張結果圖片
os.makedirs("outputs", exist_ok=True)

# 讀取圖片
img = cv2.imread("lena.png").astype(np.float32)
```

2. 分離 RGB 通道：用 NumPy 切片，取得每個顏色的單通道矩陣，並各自輸出成灰階 R, G, B 圖。

```
# 分離 R, G, B
B = img[:, :, 0]
G = img[:, :, 1]
R = img[:, :, 2]

# 儲存灰階 R, G, B
cv2.imwrite("outputs/R.png", R)
cv2.imwrite("outputs/G.png", G)
cv2.imwrite("outputs/B.png", B)
```

3. 計算 YUV：計算完成後，用 `np.clip()` 把值限制在 `[0,255]`，並轉為 `uint8`。

```
# -----
# RGB → YUV
# -----
Y = 0.299 * R + 0.587 * G + 0.114 * B
U = -0.169 * R - 0.331 * G + 0.5 * B + 128
V = 0.5 * R - 0.419 * G - 0.081 * B + 128

# 限制範圍 [0,255]
Y = np.clip(Y, 0, 255).astype(np.uint8)
U = np.clip(U, 0, 255).astype(np.uint8)
V = np.clip(V, 0, 255).astype(np.uint8)

cv2.imwrite("outputs/Y.png", Y)
cv2.imwrite("outputs/U.png", U)
cv2.imwrite("outputs/V.png", V)
```

4. 計算 YCbCr：定義轉換矩陣與偏移量，將 (R,G,B) 展平成向量，做矩陣相乘再加上 offset，最後 reshape 回原始大小。完成後，用 np.clip() 把值限制在 [0,255]，並轉為 uint8。

```
# -----  
# RGB → YCbCr  
# -----  
CbCr_matrix = np.array([[0.257, 0.504, 0.098],  
                        [-0.148, -0.291, 0.439],  
                        [0.439, -0.368, -0.071]], dtype=np.float32)  
  
offset = np.array([16, 128, 128], dtype=np.float32)  
  
# 展平成 (N,3)  
h, w = R.shape  
rgb = np.stack([R.flatten(), G.flatten(), B.flatten()], axis=1)  
  
ycbcr = np.dot(rgb, CbCr_matrix.T) + offset  
  
Y2 = ycbcr[:, 0].reshape(h, w)  
Cb = ycbcr[:, 1].reshape(h, w)  
Cr = ycbcr[:, 2].reshape(h, w)  
  
Y2 = np.clip(Y2, 0, 255).astype(np.uint8)  
Cb = np.clip(Cb, 0, 255).astype(np.uint8)  
Cr = np.clip(Cr, 0, 255).astype(np.uint8)  
  
cv2.imwrite("outputs/Cb.png", Cb)  
cv2.imwrite("outputs/Cr.png", Cr)
```

5. 實驗結果：

R.png



G.png



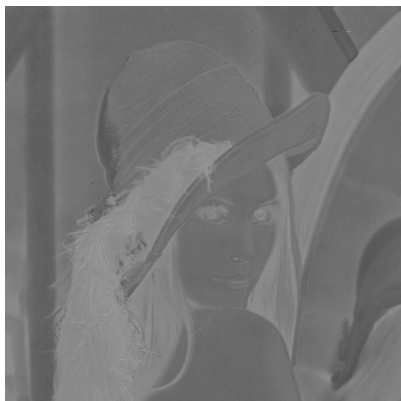
B.png



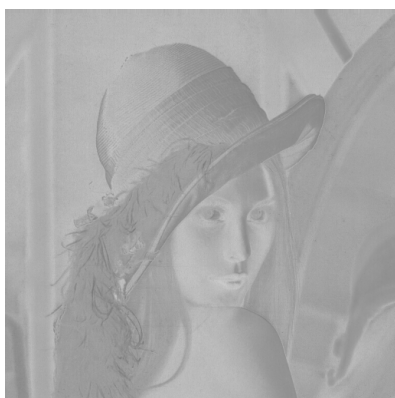
Y.png



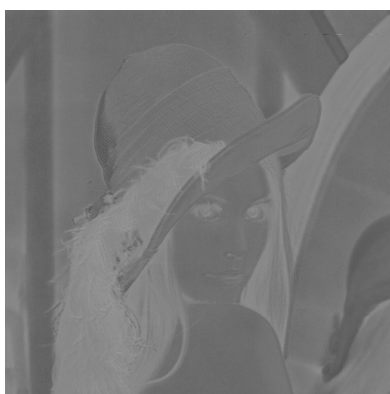
U.png



V.png



Cb.png



Cr.png



6. 結論

- 成功將 $RGB \rightarrow YUV$ 與 $RGB \rightarrow YCbCr$ 的轉換公式實作完成。
- Y 通道保留影像亮度資訊，外觀類似原始灰階圖。
- U/V 與 Cb/Cr 則表現出色彩差異，資訊較抽象，主要應用於壓縮與顏色分析。